

Monica Joya

ITAI 3377: Artificial Intelligence for Edge and IoT Devices

L03: Deploying a Simple AI Model on a Simulated Edge Device

February 7, 2026

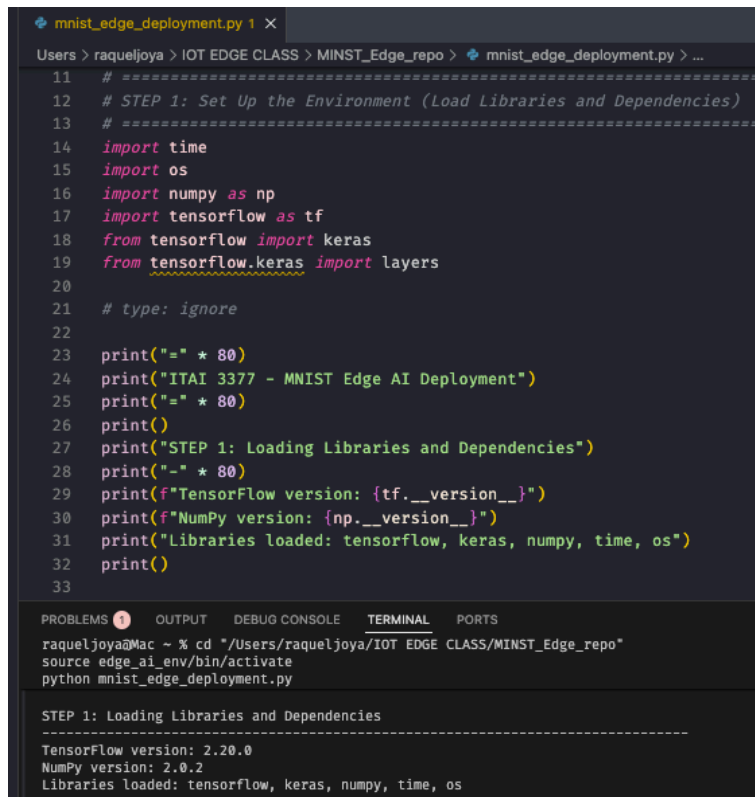
Simulation Documentation Report

Introduction

This report documents my process of deploying a simple AI model on a simulated edge device for ITAI 3377. I trained a CNN on the MNIST handwritten digit dataset, converted it to TFLite format, and deployed it to Edge Impulse to test how the model performs on a resource-constrained Arduino device. Each step is detailed below with my actual results.

1. Environment Setup

- **Hardware:** Mac M4
- **IDE:** Visual Studio Code
- **Python:** 3.9.6 (virtual environment: edge_ai_env)
- **TensorFlow:** 2.20.0
- **Node.js:** v25.6.0 / npm 11.8.0
- **Edge Impulse CLI:** v1.37.2



```
mnist_edge_deployment.py 1 X
Users > raqueljoya > IOT_EDGE_CLASS > MINST_Edge_repo > mnist_edge_deployment.py > ...
11 # =====
12 # STEP 1: Set Up the Environment (Load Libraries and Dependencies)
13 # =====
14 import time
15 import os
16 import numpy as np
17 import tensorflow as tf
18 from tensorflow import keras
19 from tensorflow.keras import layers
20
21 # type: ignore
22
23 print("=" * 80)
24 print("ITAI 3377 - MNIST Edge AI Deployment")
25 print("=" * 80)
26 print()
27 print("STEP 1: Loading Libraries and Dependencies")
28 print("-" * 80)
29 print(f"TensorFlow version: {tf.__version__}")
30 print(f"NumPy version: {np.__version__}")
31 print("Libraries loaded: tensorflow, keras, numpy, time, os")
32 print()
33
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
raqueljoya@Mac ~ % cd "/Users/raqueljoya/IOT_EDGE_CLASS/MINST_Edge_repo"
source edge_ai_env/bin/activate
python mnist_edge_deployment.py

STEP 1: Loading Libraries and Dependencies
-----
TensorFlow version: 2.20.0
NumPy version: 2.0.2
Libraries loaded: tensorflow, keras, numpy, time, os
```

2. Dataset Preparation

The MNIST dataset was loaded via TensorFlow's Keras API: 60,000 training images and 10,000 test images of handwritten digits (0–9), each 28×28 pixels in grayscale. Pixel values were normalized from 0–255 to 0–1. Images were reshaped to (28, 28, 1) for the convolutional layers.

```
32 # =====
33 # STEP 2: Prepare the Dataset
34 # =====
35 print("STEP 2: Loading and Preprocessing MNIST Dataset")
36 print("-" * 80)
37
38 # Load MNIST dataset
39 (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
40
41 print(f"Original training set shape: {x_train.shape}")
42 print(f"Original test set shape: {x_test.shape}")
43
44 # Normalize pixel values to 0-1 range
45 x_train = x_train.astype("float32") / 255.0
46 x_test = x_test.astype("float32") / 255.0
47
48 # Reshape to 28x28x1 (add channel dimension for CNN)
49 x_train = np.expand_dims(x_train, -1)
50 x_test = np.expand_dims(x_test, -1)
51
52 print(f"Preprocessed training set shape: {x_train.shape}")
53 print(f"Preprocessed test set shape: {x_test.shape}")
54 print(f"Training samples: {len(x_train)}, Test samples: {len(x_test)}")
55 print(f"Pixel value range: [{x_train.min()}, {x_train.max()}]")
56 print()
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
raqueljoya@Mac ~ % cd "/Users/raqueljoya/IOT_EDGE_CLASS/MNIST_Edge_repo"
source edge_ai_env/bin/activate
python mnist_edge_deployment.py
```

STEP 2: Loading and Preprocessing MNIST Dataset

```
-----
Original training set shape: (60000, 28, 28)
Original test set shape: (10000, 28, 28)
Preprocessed training set shape: (60000, 28, 28, 1)
Preprocessed test set shape: (10000, 28, 28, 1)
Training samples: 60000, Test samples: 10000
Pixel value range: [0.0, 1.0]
```

3. Model Architecture and Training

Layer	Type	Details
1	Conv2D	32 filters, 3×3, ReLU
2	MaxPooling2D	2×2 pool size
3	Flatten	2D to 1D
4	Dense	128 neurons, ReLU
5	Dense	10 neurons, Softmax

```
raqueljoya@Mac ~ % cd "/Users/raqueljoya/IOT_EDGE_CLASS/MINST_Edge_repo"
source edge_ai_env/bin/activate
python mnist_edge_deployment.py

STEP 3: Building and Training CNN Model
-----
Model: "MNIST_CNN"



| Layer (type)                 | Output Shape       | Param # |
|------------------------------|--------------------|---------|
| conv2d (Conv2D)              | (None, 26, 26, 32) | 320     |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0       |
| flatten (Flatten)            | (None, 5408)       | 0       |
| dense (Dense)                | (None, 128)        | 692,352 |
| dense_1 (Dense)              | (None, 10)         | 1,290   |



Total params: 693,962 (2.65 MB)
Trainable params: 693,962 (2.65 MB)
Non-trainable params: 0 (0.00 B)

Model compiled successfully!
Optimizer: Adam
Loss: Sparse Categorical Crossentropy
Metrics: Accuracy
```

Compiled with Adam optimizer and sparse categorical crossentropy loss. Trained for 5 epochs in 18.57 seconds.

Epoch	Training Accuracy	Validation Accuracy
1	87.96%	96.69%
2	97.95%	98.23%
3	98.76%	98.53%
4	99.09%	98.23%
5	99.34%	98.59%

```

Training model for 5 epochs...
-----
Epoch 1/5
469/469 4s 8ms/step - accuracy: 0.8796 - loss: 0.4283 - val_accuracy: 0.9669 - val_loss: 0.1040
Epoch 2/5
469/469 4s 8ms/step - accuracy: 0.9795 - loss: 0.0695 - val_accuracy: 0.9823 - val_loss: 0.0564
Epoch 3/5
469/469 4s 8ms/step - accuracy: 0.9876 - loss: 0.0416 - val_accuracy: 0.9853 - val_loss: 0.0431
Epoch 4/5
469/469 4s 8ms/step - accuracy: 0.9909 - loss: 0.0305 - val_accuracy: 0.9823 - val_loss: 0.0518
Epoch 5/5
469/469 4s 7ms/step - accuracy: 0.9934 - loss: 0.0226 - val_accuracy: 0.9859 - val_loss: 0.0438
-----
Training completed in 18.57 seconds

```

4. TFLite Conversion

Metric	Value
Original Keras Model (.h5)	8.16 MB
TFLite Model (.tflite)	2.71 MB
Size Reduction	66.77%

```

113 # =====
114 # STEP 4: Convert and Deploy the Model
115 # =====
116 print("STEP 4: Convert and Deploy the Model")
117 print("-" * 80)
118
119 # Save the full Keras model
120 keras_model_path = "mnist_model.h5"
121 model.save(keras_model_path)
122 keras_model_size = os.path.getsize(keras_model_path)
123 print(f"Keras model saved: {keras_model_path}")
124 print(f"Keras model size: {keras_model_size:,} bytes ({keras_model_size / 1024:.2f} KB)")
125 print()
126
127 # Convert to TFLite format
128 print("Converting to TFLite format...")
129 converter = tf.lite.TFLiteConverter.from_keras_model(model)
130 tflite_model = converter.convert()
131
132 # Save TFLite model
133 tflite_model_path = "model.tflite"
134 with open(tflite_model_path, "wb") as f:
135     f.write(tflite_model)
136
137 tflite_model_size = os.path.getsize(tflite_model_path)
138 print(f"TFLite model saved: {tflite_model_path}")
139 print(f"TFLite model size: {tflite_model_size:,} bytes ({tflite_model_size / 1024:.2f} KB)")
140 print()
141
142 # Calculate size reduction
143 size_reduction = ((keras_model_size - tflite_model_size) / keras_model_size) * 100
144 print(f"Size reduction: {size_reduction:.2f}%")
145 print()

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

raqueljoya@Mac ~ % cd "/Users/raqueljoya/IOT_EDGE_CLASS/MINIST_Edge_repo"
source edge_ai_env/bin/activate
python mnist_edge_deployment.py
TFLite model saved: model.tflite
TFLite model size: 2,778,572 bytes (2713.45 KB)

Size reduction: 66.77%

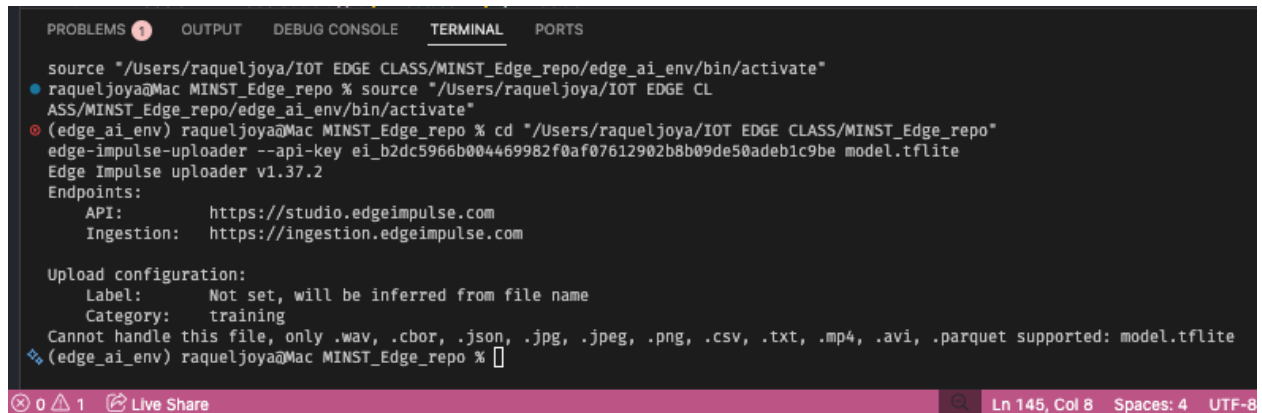
```

5. Deployment to Edge Impulse

CLI Upload Attempt:

```
edge-impulse-uploader --api-key <api-key> model.tflite
```

Error: CLI only supports data file formats, not .tflite model files:

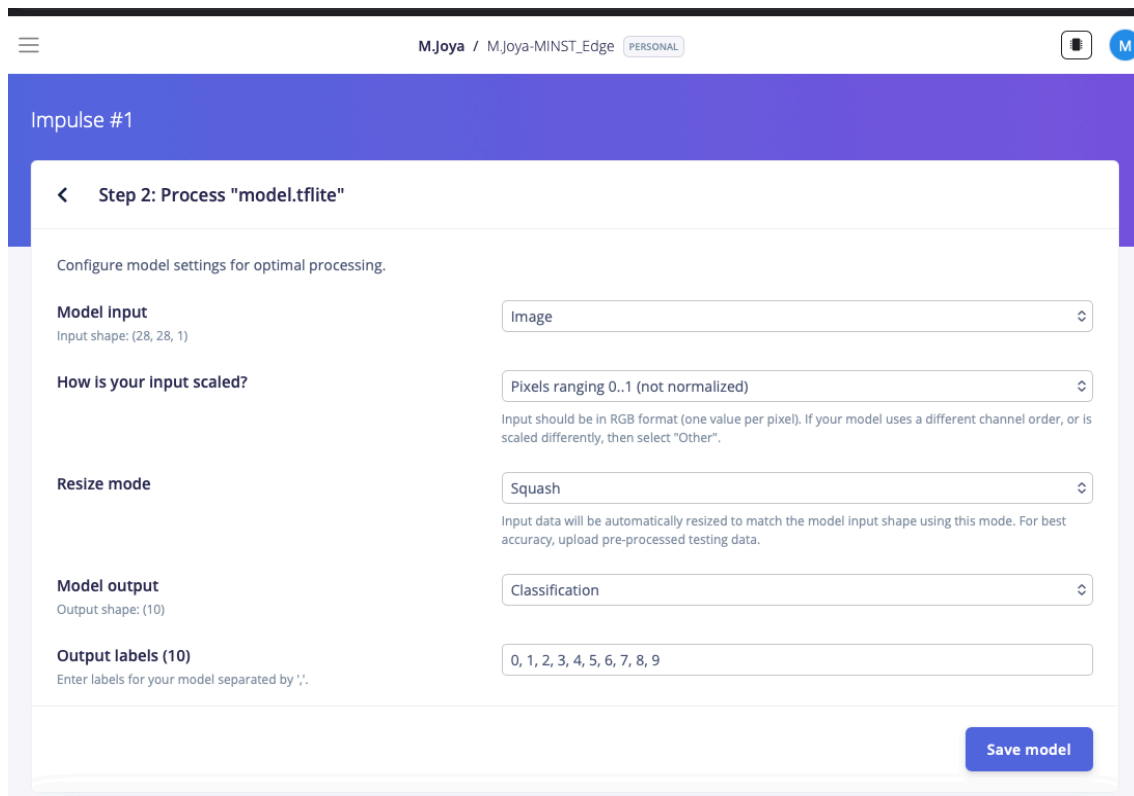


```
source "/Users/raqueljoya/IOT_EDGE_CLASS/MINST_Edge_repo/edge_ai_env/bin/activate"
raqueljoya@Mac MINST_Edge_repo % source "/Users/raqueljoya/IOT_EDGE_CLASS/MINST_Edge_repo/edge_ai_env/bin/activate"
raqueljoya@Mac MINST_Edge_repo % cd "/Users/raqueljoya/IOT_EDGE_CLASS/MINST_Edge_repo"
edge-impulse-uploader --api-key ei_b2dc5966b004469982f0af07612902b8b09de50adeb1c9be model.tflite
Edge Impulse uploader v1.37.2
Endpoints:
  API:      https://studio.edgeimpulse.com
  Ingestion: https://ingestion.edgeimpulse.com

Upload configuration:
  Label:      Not set, will be inferred from file name
  Category:   training
Cannot handle this file, only .wav, .cbor, .json, .jpg, .jpeg, .png, .csv, .txt, .mp4, .avi, .parquet supported: model.tflite
❖ (edge_ai_env) raqueljoya@Mac MINST_Edge_repo %
```

Web Interface Upload:

Model uploaded successfully through Edge Impulse Studio. Project: M.Joya-MINST_Edge. Configured as image classification with input (28, 28, 1), pixel scaling 0–1, and labels 0–9.



The screenshot shows the Edge Impulse Studio web interface for a project named "M.Joya / M.Joya-MINST_Edge". The interface is titled "Impulse #1" and shows "Step 2: Process 'model.tflite'". The configuration is as follows:

- Model input:** Image (Input shape: (28, 28, 1))
- How is your input scaled?** Pixels ranging 0..1 (not normalized) (Input should be in RGB format (one value per pixel). If your model uses a different channel order, or is scaled differently, then select "Other".)
- Resize mode:** Squash (Input data will be automatically resized to match the model input shape using this mode. For best accuracy, upload pre-processed testing data.)
- Model output:** Classification (Output shape: (10))
- Output labels (10):** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (Enter labels for your model separated by ',')

A "Save model" button is located at the bottom right of the configuration panel.

6. Testing and Validation

Target device: Arduino Nano 33 BLE Sense (Cortex-M4F 64MHz). Engine: EON™ Compiler (RAM optimized).

Metric	Result
Inferencing Time	3,695 ms
Peak RAM Usage	66.6 KB
Flash Usage	2.7 MB

Model Behavior Test:

A handwritten digit image was uploaded to test classification. The model correctly identified the digit “6”:

Label	Probability
6	0.87 (87%)
3	0.12 (12%)
0, 1, 2, 4, 5, 7, 8, 9	0.00 each

The model correctly classified the handwritten digit with high confidence on the simulated edge device...

On-device performance 

Engine:  EON™ Compiler (RAM optimized) 

 INFERRING TIME
3695 ms.

 PEAK RAM USAGE
66.6K

 FLASH USAGE
2.7M

Check model behavior

Upload test data to ensure correct model settings and proper model processing. (Optional)

Upload an image

Upload an image to try out your model. The image will be automatically resized to 28x28 (Grayscale).

Choose File  handwrittenimage_samples.jpeg

Test sample

Results



INDEX	LABEL	PROBABILITY
0	0	0.00
1	1	0.00
2	2	0.00
3	3	0.12
4	4	0.00
5	5	0.00
6	6	0.87
7	7	0.00
8	8	0.00
9	9	0.00

 Modify model settings if results do not reflect expected model behavior; otherwise save and deploy your model

Conclusion

This assignment walked me through the full process of taking an AI model from training to edge deployment. The CNN achieved 99.34% training accuracy, and after converting to TFLite with a 66.77% size reduction, I was able to deploy and test it on a simulated Arduino Nano 33 BLE Sense via the Edge Impulse platform. The model correctly classified a handwritten digit with 87% confidence, confirming it works within the constraints of a small edge device. A more detailed reflection on what I learned from each step is included in my reflective journal.